HAI913I · Évolution et restructuration des logiciels

TP 1 · Annexe 1 — Du code source au graphe d'appel

Année 2026–2027 Enseignant Bachar Rima

À QUOI SERT CETTE ANNEXE
Elle décrit l'**architecture** d'un analyseur statique comme celui du TP 1 : les composants, ce que chacun reçoit et produit, et comment vérifier chaque maillon avant de passer au suivant. L'usage de JDT est dans l'annexe 2.

1. La chaîne en une page

Le CM 02 décrit toute analyse statique en deux temps : un **extracteur de modèle** transforme le code en une représentation manipulable (le **modèle**), puis des **extracteurs de propriétés** y lisent ce qui intéresse l'analyse (les **propriétés**), que des **traitements** exploitent. Le TP 1 ajoute un maillon souvent oublié : un **modèle de faits** à vous, qui découple les traitements de la bibliothèque d'analyse.

COMPOSANT	REÇOIT	PRODUIT	ÉTAPES
Entrée	arguments de la ligne de commande	chemin du projet, action, seuil X validés	B4
Extracteur de modèle	racine des sources, fichiers `.java`	un `CompilationUnit` par fichier, avec *bindings*	A0
Extracteurs de propriétés	les AST	structure, appels, receveurs (via des visiteurs)	A1 A2 A3
Modèle de faits	les propriétés extraites	vos objets : types, attributs, méthodes, appels	B1
Traitements	le modèle de faits	métriques ; graphe d'appel	B2 B3
Sorties	résultats des traitements	texte, CSV, JSON, DOT…	B3 B4

POURQUOI UN MODÈLE DE FAITS ?
- **Découplage** : les métriques et le graphe ne dépendent plus de JDT. Refaire l'extraction avec Spoon (partie C) ne touche pas aux traitements.
- **Test** : un traitement se teste sur un petit modèle construit à la main, sans analyser de fichier.
- **Réutilisation** : le TP 2 calcule un couplage entre classes à partir des appels ; il repartira de ce modèle, pas de l'AST.

2. À quoi ressemble concrètement un modèle de faits ?

L'étape B1 fixe ce que le modèle doit permettre : retrouver les chiffres de A2 et A3 sans réanalyser le code, alimenter B2 et B3 sans aucune classe de JDT, produire une sortie identique d'une exécution à l'autre. Elle n'en fixe pas la forme. La figure 1 en montre une, en cinq classes.

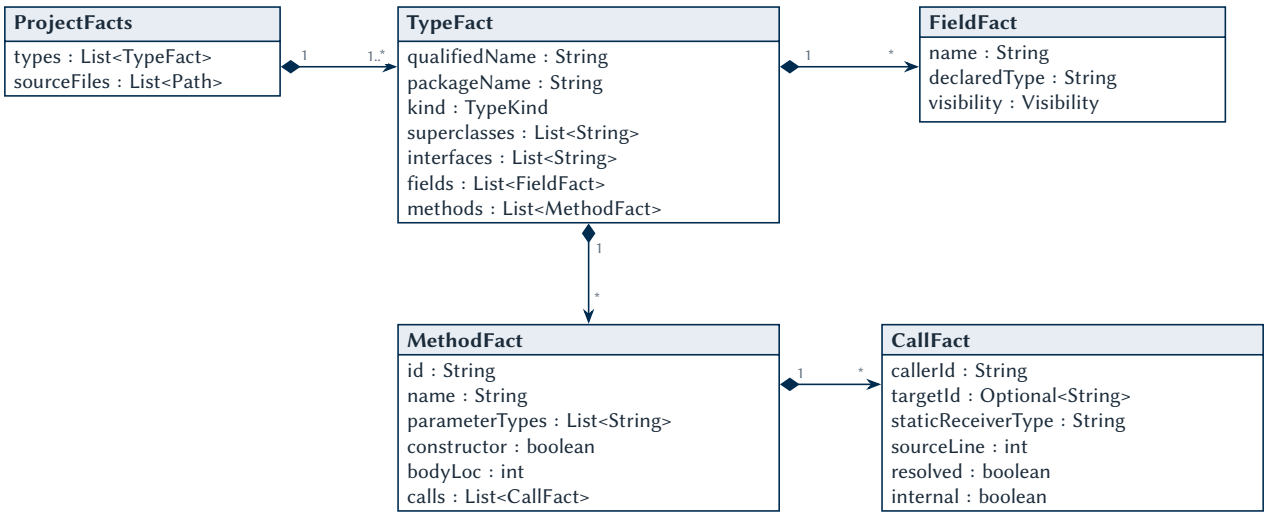


Figure 1 – Un modèle de faits possible (illustration). Chaque attribut de type liste matérialise l'association dessinée ; aucune classe de JDT n'apparaît.

UN EXEMPLE

Ceci est un exemple de représentation, pas une API imposée. Vous pouvez utiliser des *records*, classes, collections ou identifiants différents, à condition de satisfaire les propriétés du sujet. Vos choix se justifient dans le compte-rendu (étape B1).

Deux choix de cet exemple méritent d'être remarqués :

- méthodes et appels sont reliés par des **identifiants textuels** stables (type qualifié, nom, types des paramètres), jamais par des objets JDT : `library.service.Catalog#add(library.model.Item)` distingue les deux surcharges de `add` ;
- un appel non résolu reste un fait (`resolved` faux, pas de cible) : il est compté, mais ne produit pas d'arc.

2.1. Un fait, du code source à ses usages

Prenons l'appel du point de contrôle A3 : `loanable.checkOut(member)`, ligne 36 de `LoanService.borrow`.

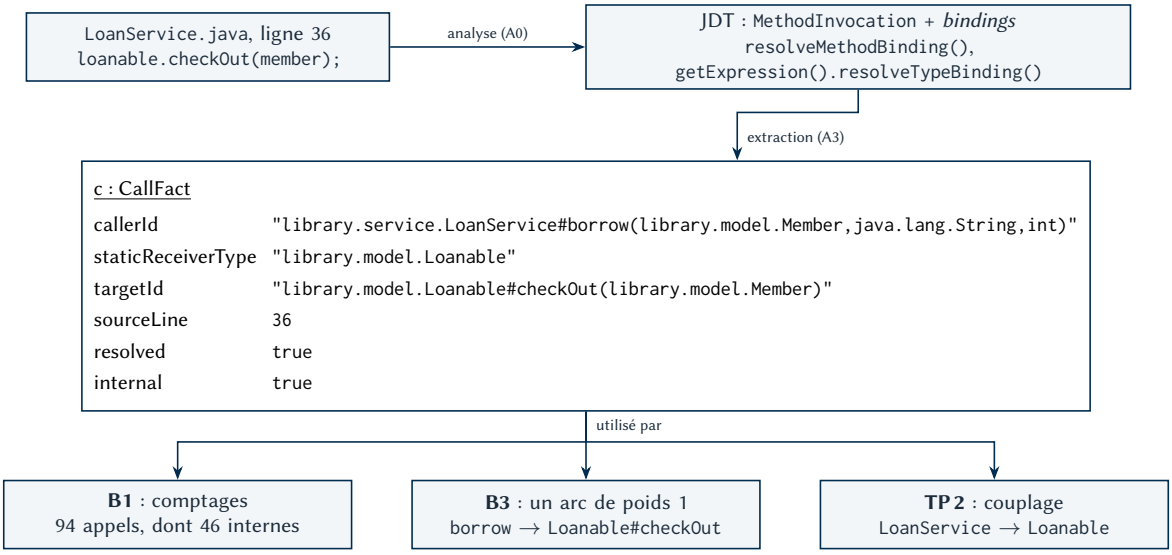


Figure 2 – Un fait du projet de validation : lu dans l'AST de JDT pendant l'extraction, puis utilisé sans JDT.

Le receveur enregistré est `Loanable`, type statique de l'expression `loanable`, et non `Item`, classe de l'objet réel. La cible est donc la méthode de l'interface : c'est pourquoi `Item#checkOut(Member)` n'a aucune appelante au point de contrôle B3.

2.2. Où s'arrête JDT

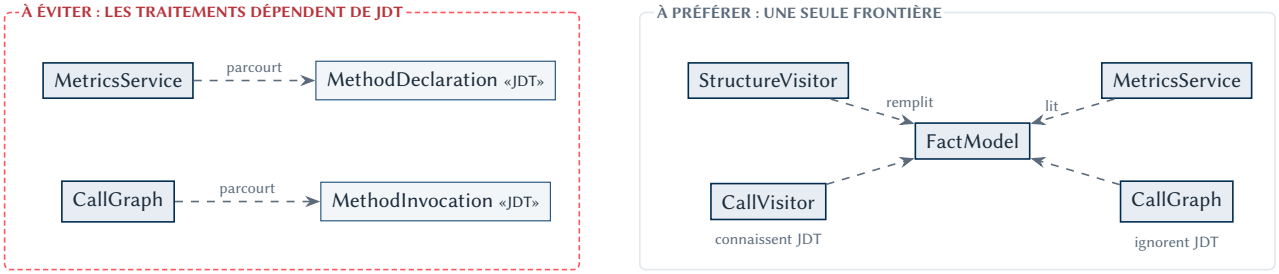


Figure 3 – À gauche, à éviter : chaque traitement relit lui-même des nœuds de JDT. À droite, à préférer : seuls les visiteurs connaissent JDT, les traitements ne lisent que le modèle de faits.

À gauche, remplacer l'extraction par Spoon (partie C), tester une métrique sur un modèle écrit à la main ou réutiliser les appels au TP 2 oblige à retoucher chaque traitement. À droite, les visiteurs traduisent l'AST en faits une fois pour toutes ; tout ce qui suit ne voit que le modèle, comme l'exige B1. L'annexe 3 (section « Utiliser un visiteur dans la chaîne du TP ») montre plusieurs façons d'écrire ces visiteurs.

3. Une organisation possible

```
hai913i.tp1
|-- cli        lit les arguments, affiche l'usage, enchaine les etapes, codes de sortie
|-- parse      trouve la racine des sources et les fichiers ; configure ASTParser
|-- extract    visiteurs JDT -> objets du modele            (depend de parse et model)
|-- model      types, attributs, methodes, appels           (ne depend de rien)
|-- metrics    les 13 metriques, a partir du modele         (depend de model)
|-- callgraph  construction et requetes du graphe           (depend de model)
`-- report     mise en forme texte / CSV / JSON / DOT       (depend de model, metrics, callgraph)
```

- **Une seule classe connaît la configuration d'`ASTParser`.** Le reste du programme lui demande « les unités de compilation de ce projet », sans savoir comment elles sont obtenues : c'est une *façade* au sens du GoF, qui cache une API riche derrière une opération simple.
- **Un visiteur par préoccupation** (structure, appels) plutôt qu'un visiteur qui fait tout : chacun reste court et testable.
- **La ligne de commande orchestre**, elle ne calcule rien. Si elle dépasse une centaine de lignes, une responsabilité s'y est glissée.

4. Le parcours, étape par étape

ÉTAPE	CE QU'ON CONSTRUIT	COMMENT SAVOIR QUE C'EST JUSTE
A0	configuration de l'analyseur, lecture du projet	14 unités, 0 erreur ; un appel entre deux fichiers est résolu
A1	un visiteur qui affiche l'arbre	les nœuds attendus pour `Dvd.java` ; `Category` est un `EnumDeclaration`
A2	l'extraction de la structure	15 types, 59 méthodes, 21 attributs ; les membres de `LoanService.Loan` ne sont pas comptés dans `LoanService`
A3	l'extraction des appels et des receveurs	94 appels, 0 non résolu ; receveur `Loanable` pour `loanable.checkOut(. . .)`
B1	le modèle de faits	mêmes chiffres, calculés depuis le modèle ; sortie identique d'une exécution à l'autre
B2	les métriques	tableau du point de contrôle B2
B3	le graphe d'appel	59 nœuds, 44 arcs, 46 sites ; aucun appelant pour `Item#checkOut`
B4	la ligne de commande et les erreurs	échantillons de `resources/robustness` ; copie fraîche

5. Valider au fil de l'eau

1. **Automatisez les points de contrôle.** Un test JUnit qui analyse `resources/validation` et vérifie 15 types, 59 méthodes, 94 appels coûte quelques minutes et détecte immédiatement une régression.
2. **Testez les cas limites sur de petits extraits.** JDT accepte une chaîne de caractères comme source (`setSource(char[])`) : un test de trois lignes suffit pour vérifier un appel via `super`, une lambda ou une surcharge.
3. **Testez les traitements sans JDT.** Construisez à la main un modèle de quatre classes et vérifiez les « 10 % », une classe sans méthode, $n = 15$.
4. **Passez les échantillons de robustesse** avant chaque rendu intermédiaire.
5. **Ne réglez rien sur le projet de validation.** Les correcteurs exécutent votre outil sur un second projet aussi.

6. Symptômes fréquents

SYMPTÔME	CAUSE PROBABLE	REMÈDE
14 types au lieu de 15	seul `visit(TypeDeclaration)` est redéfini	visiter aussi `EnumDeclaration` (et `RecordDeclaration`)
`LoanService` a 10 méthodes	visite récursive des membres : celles de `Loan` s'y ajoutent	lire `bodyDeclarations()` du type, ou tenir une pile de types
tous les *bindings* sont `null`	`setResolveBindings(true)` absent, ou mauvaise racine des sources	annexe 2, section « Quand un *binding* est nul »
environ la moitié des appels non résolus	racine des sources = dossier du projet au lieu de `src`	passer le dossier qui contient les paquetages
93 appels au lieu de 94	`SuperMethodInvocation` oublié	visiter aussi `super.m()`
receveur `Item` pour `loanable.checkOut`	type de l'objet créé, ou type déclarant de la cible	type de l'*expression* receveur
LOC de méthode trop grandes	position de la `MethodDeclaration`, qui inclut signature, annotations et Javadoc	mesurer le *corps* (`getBody()`)
« 10 % » : 1 classe au lieu de 3	`(int)(n * 0.1)`	$\lceil n/10 \rceil$ en entiers
les deux `add` de `Catalog` fusionnés	identifiant de méthode sans les paramètres	inclure les types des paramètres
sortie différente d'une exécution à l'autre	`HashMap` / `HashSet` parcourus tels quels	trier, ou utiliser des structures ordonnées

SYMPTÔME (ENVIRONNEMENT)	CAUSE	REMÈDE
`SecurityException: Invalid signature file digest` au lancement du JAR	JAR fusionné contenant les signatures des JAR Eclipse	exclure `META-INF/*.SF`, `*.RSA`, `*.DSA` (voir le `pom.xml` du `starter`)
`mvn exec:java -Dexec.args=...` échoue sur un chemin avec espaces	découpage des arguments	préférer `java -jar` avec un chemin entre guillemets
Windows : « chemin introuvable » dans un dossier profond	limite de 260 caractères des chemins	travailler dans un dossier plus court (`C:\dev\tp1`)
`UnsupportedClassVersionError`	JDK plus ancien que 17 à l'exécution	vérifier `java -version` et `JAVA_HOME`

7. Travailler en binôme

- Découpez selon la chaîne : l'un écrit un extracteur, l'autre les tests du point de contrôle correspondant — puis échangez les rôles à l'étape suivante, pour que chacun puisse expliquer tout le rendu.
- Un dépôt Git partagé, un *commit* par point de contrôle franchi.
- Écrivez le README au fur et à mesure : c'est le meilleur test de la reproductibilité, et il sert à la vidéo.

ET APRÈS LE TP 1

Le poids des arcs du graphe d'appel — le nombre de sites d'appel entre deux méthodes — est la donnée de départ du **couplage** entre classes du TP 2. Conservez-le dans votre modèle, même si le TP 1 n'en fait qu'un affichage.